Prompt for Claude: Build the "JanMat" Platform

Act as an expert Cloud Architect and LLMOps Engineer. We are building a winning proof-of-concept (POC) prototype for Track 1 of a Google AI Hackathon. The platform, named **JanMat** (People's Priority Engine), is an AI-driven constituency development planning tool.

Your goal is to write clean, modular, production-ready backend code (Python/Node.js) to deploy on Google Cloud that satisfies the entire end-to-end flow.

Part 1: Project Executive Summary & Core Mission

Most hackathon teams will build simple vector-search chatbots that summarize citizen complaints. **JanMat** wins by acting as an **Objective Decision-Support Engine** for Members of Parliament (MPs). It takes unstructured, multilingual citizen feedback (voice, text, images), normalizes it, aggregates it into geographic demand hotspots, and cross-references it against hard public datasets (Census, infrastructure availability). It outputs a ranked list of developmental projects backed by a transparent, data-grounded **Evidence Log** to eliminate bias and guesswork.

Part 2: The Prescribed Tech Stack (Strict Alignment)

We must use Google Cloud native services to maximize deployability and take advantage of the ecosystem. Do not use outside wrappers (like LangChain) unless necessary; prefer direct SDK integrations:

- **Multimodal Intake & Language:**
  - **Cloud Speech-to-Text:** Convert citizen regional voice notes into raw text.
  - **Translation API:** Translate multilingual inputs into English/Hindi for normalized backend analytical processing.
  - **Gemini Multimodal API (Vertex AI):** Extract structured parameters from text/images.
- **Data, Backend & Orchestration:**
  - **Cloud Run:** Containerized, serverless hosting for backend API orchestration (FastAPI/Express).
  - **Firebase:** Real-time state management, authentication, and hosting for the administrator frontend.
  - **BigQuery:** Multi-dataset analytical warehouse to store normalized citizen

complaints and query public government datasets.

- **Mapping & Visualization:**
  - **Google Maps Platform:** Render geographic demand clusters and coordinate pins onto an interactive visual dashboard for the MP's office.
- **Target Public Datasets (Mocked/Ingested):**
  - **data.gov.in / Census / NFHS datasets:** Existing schools per village, hospital bed metrics, distances to nearest infrastructure.

Part 3: Track 1 Core Requirements & System Flow

You must implement a three-stage architectural pipeline:

Phase 1: Structured Citizen Ingestion (Async Pipeline)

1. **Input Types:** Accepts a text message, an audio recording (Indic language), or a photo of a broken facility.
2. **Audio Processing:** Route raw audio to Cloud Speech-to-Text.
3. **Multimodal Extraction:** Pass text and photos to the Gemini API using **Structured Outputs** (JSON Schema). Force Gemini to return exactly this payload structure without markdown blocks:

```
{
 "category": "Education | Health | Roads | Water | Sanitation",
 "latitude": 12.9716,
 "longitude": 77.5946,
 "urgency_rating": 1,
 "summary_en": "Extracted english summary of the underlying problem"
}
```

4. **Storage:** Stream this clean JSON object directly into a BigQuery table named citizen_grievances.

Phase 2: The Cross-Dataset Analytical Engine (The Brain)

1. **Clustering:** Run a spatial aggregation query in BigQuery to group raw citizen complaints into "Demand Hotspots" (e.g., a 2km radius with over 50 specific water scarcity complaints).
2. **Infrastructure Grounding (RAG):** When an aggregation cluster crosses a threshold, query your public_infrastructure tables in BigQuery to extract baseline facts (e.g., "This village has zero secondary schools within an 8km radius").
3. **Algorithmic Prioritization Formula:** Combine the subjective citizen demand metric (D) and the objective gap index (G) to calculate an actionable priority score (P). Note: This

prevents dense urban populations from drowning out critical rural infrastructure needs.

Phase 3: The MP Executive Dashboard

1. Provide an API endpoint for the administrative interface that returns the ranked list of proposed projects.
2. For every single project, use Gemini to generate an **Evidence Log** summarizing why it is ranked at that level (e.g., "Ranked #1: High School Construction in Ward 4. Grounding: 120 citizen voice requests correlated with Census data showing 40% of local teenagers travel >10km for schooling.").

Part 4: Technical Guardrails against Evaluation Parameters

To secure maximum points, your code implementation must explicitly address these 4 metrics:

Metric	Hackathon Expectation	How "JanMat" Achieves It in Your Implementation
Technical Execution (25%)	AI does real, non-decorative work. End-to-end prototype functions smoothly.	We bypass raw chat interfaces. Gemini is strictly utilized as a **Deterministic Parsing and Reasoning Agent** mapping outputs to structured databases.
Deployability & Scalability (25%)	Scalable in weeks beyond pilot site; realistic runtime overhead.	Built entirely on **Serverless Architecture (Cloud Run + BigQuery)**. Infrastructure scales horizontally to support thousands of parallel citizen inputs and drops to zero when idle.
Inclusivity & Accessibility (15%)	Voice/low-literacy entry points, regional language handling.	Native pipeline implementation of **Speech-to-Text → Translation API** allowing textless voice-driven

Metric	Hackathon Expectation	How "JanMat" Achieves It in Your Implementation
		submissions from citizens.
Problem-Solution Fit (20%)	Addresses competing priorities objectively against real datasets.	Implement the **Cross-Dataset Grounding** logic where citizen voices are programmatically cross-referenced with public infrastructure metrics before ranking.

Tasks for Claude to Begin Implementing:

- **Draft the FastAPI backend structure** ready for deployment on Cloud Run.
- **Write the Vertex AI Gemini integration function** that enforces the Pydantic JSON output structure for the multi-modal parsing layer.
- **Write a mock BigQuery SQL query script** that demonstrates how we combine a spatial cluster of citizen complaints with a static public dataset table to output our priority score.